

# 2026-1 / IIC2173 - E1 | CityExpress

Fecha de entrega: 03/05/2026 - 4 Semanas

## Acerca de

Luego de que QuackPackage fuera adquirida por una empresa de *Private capital*, fue convertida en una máquina de profit para enviar paquetes de tamaño pequeño solo a nivel terrestre (cosa de no gastar en R&D).

Desilusionado con el pobre aprovechamiento de la tecnología de portales (ni se molestaron en patentarlo ni les interesaba), y con una visión poderosa de lo valioso que puede ser esto, ustedes, el antiguo departamento de R&D deciden crear CityExpress, la red de mensajería dimensional. Utilizando estos portales cada grupo establece una sucursal en cada ciudad de importancia conocida. Para simular el funcionamiento de esta red y posteriormente administrarla, se preparan a modelar un sistema distribuido de paquetería.

*Suposicion clave: recibir un mensaje respecto a un paquete es lo mismo que recibir el paquete físico a través del portal*

## Sobre las ciudades

Cada grupo para esta entrega representará una ciudad. Estas ciudades tienen códigos asociados de envío. Las ciudades disponibles son

```
{
  "cities": [
    { "code": "HGW", "name": "Hogwarts" },
    { "code": "COR", "name": "Coruscant" },
    { "code": "REE", "name": "Re-Estize" },
    { "code": "RAP", "name": "Rapture" },
    { "code": "RNC", "name": "Rancagua" },
    { "code": "TAL", "name": "Talca" },
    { "code": "LSN", "name": "Los Santos" },
    { "code": "MTI", "name": "Minas Tirith" },
    { "code": "SPR", "name": "Springfield" },
    { "code": "NNY", "name": "New New York" },
    { "code": "MET", "name": "Metropolis" },
    { "code": "KLD", "name": "King's Landing" },
    { "code": "TAR", "name": "Tar Valon" },
    { "code": "ZIN", "name": "Zion" },
    { "code": "TK3", "name": "Tokyo-3" },
    { "code": "ROM", "name": "Romdo" },
    { "code": "TRA", "name": "Trantor" }
  ]
}
```

Cada ciudad tiene una cola asignada. Para enviar mensajes a esas cola, deben utilizar el binding `city.{cityId}` donde cityId es el `code` de una ciudad.

Cada grupo implementará un puesto de envío de paquetería que cumpla con las especificaciones a continuación.

## Sobre los mensajes

Cada mensaje tiene al menos

```
{
  "idpk": "uuid",
  "msgId": "uuid",
  "type": "request",
  "timestamp": "ISO 8601"
}
```

- `msgId` lo determinan la entidad que envíe y es un UUID aleatorio
- `idpk` corresponde a lo que se llama un llave de idempotencia, la que asegura que frente a múltiples reenvíos, las operaciones sensibles no se ejecuten más de una vez. Múltiples reintentos del mismo msgId deberían tener la misma llave de idempotencia para una misma operación
  - DEBE diferir del msgId puesto que las respuestas harán referencia al msgId
- `type` identificador de mensaje para alguno de los tipos disponibles
- `timestamp` generado al momento de creación del mensaje

Si ustedes envían el mensaje, deben adjuntar su cityId (code)

```
{
  ... //campos comunes msgId, idpk, type, timestamp
  "cityId": "city-code",
  ...
}
```

## ACK y NACK

Si un servicio hace una petición a otro servicio, deben hacer un acknowledge de la petición a la cola del servicio que envió el mensaje. Este debe verse así

```
{
  ...
  "type": "ack"
  ...
}
```

Si el servicio no respeta el funcionamiento del otro servicio enviando un paquete malformado, responderá con un Not ACK, NACK

```
{
  ...
  "type": "nack"
  ...
}
```

No se esperan acciones luego de un NACK, mas que el paquete formado correctamente. Será decisión del servicio que recibe el NACK que hará con este

No se hacen ACKs de ACKs para evitar un loop de exceso de cortesía

## Sobre las rutas

Cada ciudad está conectada a otra mediante rutas. Estas rutas determinan como pueden acceder a otras ciudades y tienen un valor dinámico.

Enviando un mensaje al canal *central* con este formato

```
{
  ...
  "type": "request",
  "data": {
    "ask": "distance-table"
  },
  ...
}
```

Obtendrán una tabla de distancias como esta

```
{
  ...
  "cityId": "COR",
  "type": "distance-table",
  "data": {
    "distances": {
      "HGW": {
        "destinationCode": "HGW",
        "destinationName": "Hogwarts",
        "distance": 62763183,
        "transportCost": 9351985,
        "enabled": true
      },
      ...
      ...
      ...
      "TRA": {
        "destinationCode": "TRA",
        "destinationName": "Trantor",
        "distance": 94306517,
        "transportCost": 3363820,
        "enabled": true
      }
    }
  }
},
  "timestamp": "2026-03-30T04:36:21.120Z"
}
```

En este momento, lo único que les interesa es el campo *enabled* que dice si pueden o no enviar a dicha ciudad.

Estas distancias cambiarán con el tiempo. Recibirán un mensaje con el tipo *"distance-table"* en sus colas cada cierto tiempo con los nuevos valores que deberán usar.

Recomendamos ser muy austeros con la petición de esta tabla. Solo pidanla una vez en la inicialización de sus servicios y luego vayan recibiendo los updates

## Sobre los paquetes

Cada cierto tiempo la central enviará paquetes a cada una de sus ciudades, los cuales tienen la misión de llegar a las respectivas ciudades de destino para ser entregados

Ustedes como ciudades deben recibir paquetes en su cola y

- Recibirlos y entregarlos si son para su ciudad
- Reenviarlos a la ciudad apropiada si no son para ustedes (directamente a la cola de dicha ciudad)
- Utilizar otra ciudad para reenviar si no tienen camino directo

Estos eventos de paquetes, desde cualquier origen tienen el siguiente schema JSON

```
{
  ...
  "type": "package-transit",
  "packageBody": {
    "id": "xxxxxxxx", //uuidv4
    "deliveryStrategy": "direct",
    "maxHops": int,
    "createdAt": "2026-03-01T12:00:00Z",
    "deliverNotBefore": "2026-03-20T12:00:00Z",
    "originId": "central",
    "destinationId": "CITY",
    "metaContent": "",
    "isMetaEncrypted": false,
    "constraints": {}, //vacío por ahora
    "priorityClass": "medium",
    "payment": 0
  }
  ...
}
```

Los siguientes campos son relevantes

- maxHops: Cuantas veces más un paquete ser reenviado antes de ser declarado expirado. Si el paquete no les pertenece, deben descontar 1 a este valor y reenviarlo con el valor descontado a cualquier ciudad (aleatoria y no la anterior) con la que si tengan conexión. Si el valor recibido es 0 y el paquete no es para su ciudad, debe ser declarado como expirado. Si no pueden enviar paquetes, deben reservarlos de forma persistente hasta que puedan enviar
- deliverNotBefore: El paquete debe ser entregado desde esta fecha en adelante. Este campo importa solo a la ciudad de destino
- destinationId: Donde debe llegar el paquete

Deben hacer ACK/NACK de estos mensajes a quien se los haya enviado

## Sobre el auditor

Toda accion con un paquete debe ser reportada a la central sin falta posterior a su ejecución exitosa. Las siguientes acciones deben ser auditadas

### Reenvío

Si tienen ruta directa a la ciudad para reenviar, reporten a la central con el siguiente paquete.

```
{
  ...
  "pkgId": "packageId",
  "type": "transit",
  "data": {
    "nextCityId": "target-city-code"
  },
  ...
}
```

### Ruta imposible

Si no tienen ruta directa a la ciudad, seleccionen una ruta aleatoria que no sea la central o la ciudad desde la que les llego el paquete, y reenvien el paquete. Envíen a la central el siguiente paquete

```
{
  ...
  "pkgId": "packageId",
  "type": "transit-redirect",
  "data": {
    "nextCityId": "target-city-code"
  },
  ...
}
```

### Expirado

Si un paquete expira en su ciudad

```
{
  ...
  "pkgId": "packageId",
  "type": "expired",
  ...
}
```

### Recibido

Si un paquete es recibido por ustedes (y aun no ha sido entregado)

```
{
  ...
  "pkgId": "packageId",
  "type": "received",
  ...
}
```

### Entregado

Si un paquete está en su propiedad y puede ser entregado al cliente, al momento de entregar deben reportarlo

```
{
  ...
  "pkgId": "packageId",
  "type": "delivered",
  ...
}
```

## Requisitos

### RF (20 ptos)

Su entrega debe cumplir con lo siguiente

- **RF01 (5 ptos) (Esencial):** La aplicación debe ofrecer una vista que permita consultar todos los paquetes que hayan sido recibidos por la ciudad del grupo, incluyendo al menos:
  - Identificador del paquete
  - Ciudad de origen
  - Ciudad de destino
  - MaxHops vigente al momento del último procesamiento
  - Fecha de creación
  - Fecha mínima de entrega (deliverNotBefore)
  - Estado o resultado del procesamiento realizado por la ciudad

Para cada paquete, la interfaz debe permitir identificar claramente cuál fue la última acción realizada por la ciudad sobre él, la que podrá ser una

- **RF02 (3 ptos)** La aplicación debe ofrecer una vista que muestre el estado actual de conectividad entre la ciudad del grupo y cada una de las ciudades definidas por el enunciado. Para cada ciudad destino, se debe visualizar al menos:
  - Código de ciudad
  - Nombre de ciudad
  - Si la ruta se encuentra habilitada o deshabilitada

La vista debe reflejar la información obtenida desde la tabla de distancias entregada por la central y actualizarse cuando esta cambie.

- **RF03 (10 ptos) (Esencial):** Debe implementar el sistema de ruteo para paquetes y manejar los casos solicitados
  - Redirección de paquetes
  - Drop de paquetes por *maxHops*
  - Recepción de paquetes en su ciudad
- **RF04 (2 ptos):** La aplicación debe permitir concretar la entrega de paquetes que ya hayan llegado a la ciudad del grupo y cuyo deliverNotBefore permita su entrega. Al momento de la entrega, el sistema debe:
  - marcar el paquete como entregado
  - impedir que el mismo paquete sea entregado más de una vez
  - registrar visualmente el cambio de estado

### RNF (34 ptos)

- **RNF01 (5 ptos) (Esencial):** Deben usar un formato de Backend-Frontend separado: una API con respuestas JSON y un frontend. Esto es muy importante puesto que es crítico para las siguientes entregas.
  - El Frontend debe ser una SPA con un Framework que permita exportar el build de su frontend como artefactos estáticos basado en React o Vue. Deben usar dos repositorios distintos para manejar su backend y su frontend
  - **RNF02 (2 ptos) (Esencial):** Sus aplicaciones en backend deben estar cada una en un contenedor Docker distinto. Debe coordinarse el levantamiento mediante un docker compose optimizado para producción
  - Debe utilizar ECR sobre EC2 para desplegar
- **RNF03 (2 ptos) (Esencial):** Deben tener configuradas Budget alerts en la cuenta que ocupen como grupo, para no alejarse del Free tier de AWS y que se den cuenta si les cobran para que cierren esos servicios.
- **RNF04 (5 ptos) (Esencial):** Su API debe estar detrás de una AWS API gateway tipo REST o HTTP, con los endpoints declarados en esta. Debe asociarse un subdominio a esta (e.g. <https://api.miapp.com>) y debe tener CORS correctamente configurado.
- **RNF05 (3 ptos) (Esencial):** Su app debe ofrecer su backend y frontend utilizando HTTPS.
- **RNF06 (4 ptos) (Esencial):** Deben implementar un servicio de autenticación/autorización (auth). Este servicio puede ser en base a un servicio de terceros como Auth0 (recomendado) o Cognito utilizando el setup con JWK estándar
- **RNF07 (3 ptos):** Su API Gateway debe poder usar al servicio del RNF06 para autenticar a los usuarios directamente, es decir que antes de enviar la request a su API, API Gateway verifica que el token sea correcto. Dentro de API Gateway deben crearle un Custom Authorizer si usan tipo REST para poder autenticar sus requests previos a mandarlos a su API.
- **RNF08 (3 ptos):** Su frontend debe estar desplegado en S3 con una distribución Cloudfront.
- **RNF09 (5 ptos):** Deben implementar monitoreo utilizando un proveedor de monitoreo SaaS. Se recomienda New Relic puesto que su free tier es generoso.
  - Deben tener APM básico y poder ver las requests que van pasando por el controller HTTP de su app por lo menos
  - Debe habilitarse el monitoreo de infraestructura
- **RNF10 (2 ptos):** Deben implementar las siguientes medidas de resiliencia
  - Sus containers deben levantarse al caer
  - Deben hacer retry de cada petición al broker usando un delay fibonacci.

# Documentación (6 ptos)

---

Todos estos documentos los deben dejar dentro de su repositorio de Github en una carpeta /docs .

- *RDOC01 (3 ptos)*: Deben crear un diagrama UML de componentes de la entrega, con explicaciones y detalle sobre el sistema. Esto deben tenerlo para la fecha final de entrega.
- *RDOC02 (2 ptos)*: Deben documentar los pasos necesarios para replicar la instalación y seguimiento de su flujo de Monitoreo (que pasos siguieron y como se ve la información)
- *RDOC03 (1 ptos)*: Deben dejar una documentación de alguna forma de correr su aplicación en un ambiente local para propósitos de testeo (que instalar, que poner en el .env, como correr la app, etc).

## Meta: Sobre la entrega

---

- NO se revisaran entregas que no estén en la nube bajo ningún concepto.
- Deben tener todos los requisitos no variables de la E0 para esta entrega o NO se les corregirá.
- Para esta entrega y el resto del proyecto les pediremos que creen un repositorio personal y que luego integren a su ayudante respectivo con accesos de administrador.

Al final de la entrega, la idea es que se pongan de acuerdo con su ayudante para agendar una hora y hacer una demo en vivo para su corrección. Se revisarán algunas features esenciales de código (conexión al broker).

## Del uso de agentes de AI

Pueden usar sin restricciones

- Tab completion
- Consultar via chat con sus agentes

Si usan cualquier flujo de chat on AI, deben

- Documentar **TODOS** los prompts que utilizaron y sus respuestas en flujos legibles y accesibles en archivos markdown (docs/prompts) por usuario. Deben generar un reporte resumen de cada flujo de prompts
- Todos los miembros deben estar preparados para responder preguntas sobre todo el codebase.

Si opcionalmente usan programación agéntica, esto es, cualquier interacción que permita a una instancia de AI escribir código directamente en el codebas:

- Deben adscribir a usar el sistema GSD o BMAD y utilizarlo para *TODO* el ciclo de desarrollo. Como evidencia de esto deben verse todos los artefactos usados por cualquiera de estos sistemas
  - Deben tener un sistema de milestones, con un sistema claro de planificación, ciclos y estructura
    - Antes de comenzar el desarrollo debe estar todo el roadmap documentado y los milestones de desarrollo documentados, así como las listas de requerimientos
  - Deben construir un sistema de harnessing para el uso de estos agentes, basado en spec driven development
  - *TODO* el equipo debe usar esta metodología
  - Verificabilidad: su código debe tener un coverage de al menos 75% en tests unitarios y setup listo para tests E2E
  - AI usage logs obligatorios
    - Cada interacción relevante con agentes debe quedar registrada: Prompt usado -> output generado -> decisión tomada
    - Deben documentar y justificar cada decision tomada en una nota adjunta a la decisión con tradeoffs
    - Estos logs deben ir en un documento MD por cada conversacion
- Se les pedirá un gitflow impecable con el uso de estos agentes.
  - Debe haber un flujo documentado de PRs revisados por al menos *2 de sus compañeros* a su *main trunk* por PR
  - Deben tener branch rules seteadas para
    - main protegido
  - Cada PR debe tener una explicación completa escrita por ustedes de como funciona ese código, y como se verificó su uso
- Deben notificar a su ayudante que van a trabajar con agentes de AI

Todas estos puntos son clave para que no programen sin estructura y perjudiquen el avance o colaboracion grupal. La documentación pasará a pesar un 40% como nota del proyecto.

Si utilizan programación agéntica pero no cumplen con los puntos expuestos se considerará su entrega como nula (1.0). Esta es una decisión binaria: o cumplen con todos los puntos o no cumplen.

**Si se detecta programación agéntica y no fue declarada ante su ayudante, se considerará plagio con todas las consecuencias que esto conlleva**

La información más completa la encontrarán en las documentaciones oficiales y preguntando a sus ayudantes / profesores. Si utilizan código copiado de una plataforma deben citarlo en el README.md, y pregunten a su ayudante antes de usarlo.

## Recomendaciones

---

- Comiencen la entrega lo antes posible, puesto que es más sencillo ir trabajando de a partes y seguro tendrán dudas. Se les dio plazo extra para que se adecuen a sus equipos de trabajo.
- Planifiquen con antelación: pregunten dudas o ambigüedades a sus ayudantes.
- Mucho cuidado con los deploys a última hora, la maldición de la demo es muy real.
- Ocupen el Free Tier de AWS, que tiene capacidad para todos estos requerimientos. Recibirán 100 usd por instancia y 20 usd por completar ciertas metas sencillas.
- NO está planificado hacer devolución por uso de dolares en AWS. Para la entrega el free tier de AWS es suficiente para conseguir todos los puntos. En caso de utilizar dólares en su solución, el curso no puede hacerles devolución de estos bajo ninguna causa. Si tienen problemas con costos que no saben como detener háblenlos con su ayudante designado.
- Usen una cuenta nueva o de alguien que no tenga otras cargas en AWS, para evitar cargos por ahora , además de usar una tarjeta prepago y los budget alerts de AWS para evitar costos ocultos.
- Pueden usar librerías para generar el gráfico de precios semanales

## Descuentos

---

Está totalmente prohibido subir su archivo .env o .pem al repositorio, en caso de subir su archivo .env se les descontará 5 decimas, y en el caso del .pem no se corregirá su entrega.

## Sobre abuso de la mensajería.

Si un grupo por alguna razón introduce loops de mensajes infinitos contaminando el sistema de mensajería o envía mensajes deliberadamente maliciosos, sucederá lo siguiente

- Se le notificará amablemente que corrija el problema en 8 horas
- Se le impondrá un ban de 24 horas para operar en el broker en primera instancia.
- Luego de 72 horas y 5 ptos de dcto
- Luego de 1 semana y 10 ptos de dcto
- Luego de 3 semanas y 30 ptos de dcto

El ban puede extenderse más alla de la fecha de entrega de una entrega en específico

## Consideraciones

No se considerarán entregas:

- Con componentes que corran en sus computadores o servidores que no sean los básicos de AWS. Algunos ejemplos, los servicios de AWS serían:
  - EC2
  - VPC
  - IAM
  - S3
  - Lambda
- Montadas en Heroku/Firebase/Elastic beanstalk/Lightsail/Netlify o PaaS similares.
- Que no estén documentadas.

Pueden utilizar cualquiera de los siguientes lenguajes. Los frameworks son los recomendados. Sugerimos que utilizen en cada grupo el lenguaje que todos sepan usar más o les parezca más sencillo de aprender. Los lenguajes en cursiva poseen ayudantes capaces de ayudarlos en su implementación de código.

- Python
  - FastAPI
  - Django
- Javascript y typescript
  - Koa
  - Express
  - NestJS (recomendado si han tocado typescript)
- Ruby
  - Rails En frontend, con el sistema de estado que prefieran
- Javascript
  - React SPA
  - Vue
  - Next CSR
  - Nuxt

## Puntaje

### Atraso

Para esta entrega se les descontará 0.5 puntos en la nota máxima por horas Fibonacci con F1 = 6 y F2 = 6.

Se considerará como atraso cualquier modificación en features o implementación que tenga que ver solo con lo que se pide en esta entrega.

| Fibonacci | Hora              | Nota maxima |
|-----------|-------------------|-------------|
| 6         | 0:01 - 5:59       | 6.5         |
| 6         | 6:00 - 11:59      | 6           |
| 12        | 12:00 - 23:59     | 5           |
| 18        | 24:00 - 41:59     | 4.5         |
| 30        | 42:00 - 71:59     | 4           |
| ...       | 72:00 en adelante | 1           |

### Grupal

La nota se calcula como:

$$E_{1 \text{ grupal}} = 1 + E_{1 \text{ RF}} + E_{1 \text{ RNF}} + E_{1 \text{ RDOC}}$$

Siendo  $E_{1RF}$  el puntaje ponderado de los requisitos funcionales, y  $E_{1RNF}$  el correspondiente a los requisitos no funcionales y  $E_{1RDOC}$  de la documentación.

Individual

Segun el programa del curso<sup>5</sup>, esto se evalua como:

$E_1 = 1 + ((E_{1\text{ grupo}} - 1) * F_g)$

Donde  $F_g$  es un factor de coevaluación asignado por el grupo que va de 0.25 a 1.25. Para esto se enviará un form de coevaluación donde cada integrante deberá evaluar a sus compañeros de grupo con una puntuación entre 1 y 5.

| Puntaje | factor |
|---------|--------|
| 1       | 0.25   |
| 2       | 0.5    |
| 3       | 0.75   |
| 4       | 1      |
| 5       | 1.25   |

Si añaden puntos a un compañero, significa que alguien trabajó menos, por lo que los puntos deben salir de otra evaluación

De no realizar la coevaluación, asumiremos que se le entregó el mismo puntaje de coevaluación a cada integrante, es decir 4 puntos. También si la coevaluación no suma bien los puntos, se considerará que no la respondieron.

09F911029D74E35BD84156C5635688C0